Visual Computing

Janis Hutz
https://janishutz.com

October 12, 2026

1 Computer Vision

1.1 Digital Image

Def **Image** $f : \mathbb{R}^n \to S$, for greyscale $n = 2$, $S = \mathbb{R}^+$, in digital form: $I : \{1, \dots, X\} \times \{1, \dots, Y\} \to S$

Rem Pinhole size affects blur, ***Lens Camera*** as sol.

Rem Charge Coupled Device (CCD) sensor reads line-by-line (rolling shutter), then each line pixel-by-pixel

Def **Sensor array** is an array of *photosites* (bucket of el. charge $\propto$ light intensity), then read via ADC.

Rem **Blooming** Caused by photosite saturation

Def **Dark Current** CCDs give non-zero output in darkness, fluctuates randomly

Def **CMOS Sensor** Each photo sensor has own amplifier, more noise, lower sensitivity, "smart" pixels bc. CMOS

Def **Sampling** Repeated measurement on certain interval, ***Reconstruction*** is inverse. Issues: ***undersampling*** (lose information), ***oversampling*** (too large files).

Def **Nyquist Frequency** highest signal freq that can be accurately captured, half of sampling rate

Def **Quantization** Lossy discretization of analogue value, simple versions have $k = 2^b$ values for b bits

Rem Quantization "on" y-axis, Sampling "along" x-axis

Def **Res.** Geometric: PPI; Radiometric: Bits/Pixel

Def **Signal-Noise-R.** (SNR) Image quality index: $s = F \div \sigma$, with $F = \frac{1}{XY} \sum_{x=1}^{X} \sum_{y=1}^{Y} f(x, y)$.

Def **Add. Gaussian Noise** $I(x, y) = f(x, y) + c$, with $c \sim \mathcal{N}(0, \sigma^2)$, s.t. $p(c) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{c^2}{2\sigma^2}}$

Def **Poisson Noise** $p(k) = \frac{\lambda^k e^{-\lambda}}{k!}$

1.2 Filtering & Convolution

Def **Moving Average** New image G where $G[x, y]$ is average of surrounding pixels and $F[x, y]$

Def **Linear Shift-Invariant Filter.** New pixels lin. comb. of neighbours, *shift-invariant* = doing same for all pixels. Good for: smoothing, noise reduction, sharpening

Def **Linear Operation** L lin. if $L[\alpha I_1 + \beta I_2] = \alpha L[I_1] + \beta L[I_2]$, $I'_j = \sum_{i=1}^{N} \alpha_{ij} I_i$ for $j \in \{1, \dots, N\}$

Def **Linear Filtering** $I'(x, y) = \sum_{(i,j) \in N(x,y)} K(i, j) I(x + i, y + j)$ with I the input image and K the ***kernel*** of operation, $N(x, y)$ neighbourhood function for pixel (x, y). Shift-Invariant if K doesn't depend on (x, y) (i.e. same weights everywhere)

Def **Kernel** Typically matrix, center of matrix is $(0, 0)$

Def **Correlation** $I'(x, y) = \sum_{j=-k}^{k} \sum_{i=-k}^{k} K(i, j) I(x + i, y + j)$

Def **Conv.** $I'(x, y) = \sum_{j=-k}^{k} \sum_{i=-k}^{k} K(-i, -j) I(x + i, y + j)$

Gen.: $g(x) = (f * h)(x) = \int_{-\infty}^{\infty} f(\tau) h(x - \tau)\, \mathrm{d}\tau$, $h(\tau)$ kernel

Rem If $K(i, j) = K(-i, -j)$, then Convolution = Corr.

Rem Convolution is linear: $[(af + bg) * h](x) = a(f * h)(x) + b(g * h)(x)$, commutative: $(f * h)(x) = (h * f)(x)$, associative: $(f * g) * h = f * (g * h)$; Diff: $\frac{\mathrm{d}}{\mathrm{d}x}(f * h)(x) = f * h'$

Rem At edge of image, extrapolate, options: clip filter (black), wrap around, copy edge, reflect across edge

Def **Gaussian Kernel** $G_\sigma = \frac{1}{2\pi\sigma^2} e^{\frac{x^2+y^2}{2\sigma^2}}$. The smooting amount depends on σ and window size.

Rem $g(x, y) = g(x)g(y)$, with $g(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{x^2}{2\sigma^2}\right)$. Efficient implementation: First rows 1D, then cols 1D

1.3 Edge Detection

Def **Scale Space** Collection of img at diff scales & smooth.

Rem Finite diff. for $\frac{\partial f(x,y)}{\partial x} \approx \frac{f(x+1,y) - f(x,y)}{1}$, filter: $[1, -1]$. Can use par der. for edge detection

Def **Gradient** $\nabla f = [\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}]$, dir of most rapid change. ***Edge strength*** (magnitude): $M(x, y) = ||\nabla f||$.
Noise makes edge detection hard, smooth first, then edge is peak in $\frac{\partial}{\partial x}(h * f)$. ***Angle*** $\alpha(x, y) = \arctan\left(\frac{\partial f}{\partial y} \div \frac{\partial f}{\partial x}\right)$

Thm $\frac{\partial}{\partial x}(h * f) = \left(\frac{\partial}{\partial x} h\right) * f$

Def **Laplacian** $\nabla^2 f = \frac{\partial^2 f}{\partial^2 x} + \frac{\partial^2 f}{\partial^2 y}$

Def **LoG** $\nabla^2 G(x, y) = \frac{1}{2\pi\sigma^2} \left(\frac{x^2+y^2-2\sigma^2}{\sigma^2}\right) \exp\left(-\frac{x^2+y^2}{2\sigma^2}\right)$
Edge is at zero-crossing of $\left(\frac{\partial^2}{\partial^2 x} h\right) * f$. σ affects scale of edges detected (larger σ = stronger edges)

Def **Canny E.D.** ***1*** smooth w/ Gaussian, ***2*** gradient magnitude & angle, ***3*** Nonmaxima suppression to gradient mag. image, ***4*** double thresholding to detect strong and weak edge pixels, ***5*** reject weak edge pixels not connected with strong edge pixels

Def **Non-Max-Suppression** Quantize edge normal to four dir, if $M(x, y)$ smaller than either neighbour in dir, suppress, else keep

Def **Thresholding** Via thresholds $\theta_{\text{high}} \div \theta_{\text{low}} \in [2, 3]$ (typ)

1.3.1 Line Fitting

Generally challenging, missing information

Def **RANSAC** Random Sample Consensus:

1. Rand. sel. *seed group* of p. w/ base transf. est.
2. Compute transformation from seed group
3. Find *inliers* to this transformation
4. If inliers sufficiently large, recompute least-squares estimate of transformation on all inliers

Sample count: w frac of inliers, n points define hypothesis ($n = 2$ for lines), k samples chosen. Prob. sample of n points correct: w^n, prob all k samples fail: $(1 - w^n)^k$, i.e. choose k high to keep below failure rate

Def **Hough Transform** Voting technique, main idea:

1. Vote for all possible lines on which edge could lie
2. Look for line candidates that get many votes
3. Noise features votes should be inconsistent

1.4 Fourier Transform

Def **Sinusoid** $f(x) = A \cdot \sin(2\pi u x + \psi)$ with ψ phase shift, equiv: $A\cos(\psi)\sin(2\pi u x) + A\sin(\psi)\cos(2\pi u x)$

Def **Complex Exponential** $e^{i\theta} = \cos(\theta) + i\sin(\theta)$, complex conj. $e^{-i\theta} = \cos(\theta) - i\sin(\theta)$. For sinusoid: $\theta = 2\pi u x$.

Def **Cont. Time F. Ser** Sig as sum of sinusoids, $f(x) = \sum_{k=-\infty}^{\infty} c_k e^{i2\pi k x \div T}$, with T the period and c_k fourier coeff.

Def **Cont. Time F. Transf** $F(u) = \sum_{\mathbb{R}} f(x) e^{-2\pi u x}\, \mathrm{d}x$, $f(x) = \sum_{\mathbb{R}} F(u) e^{i2\pi u x}\, \mathrm{d}u$. $F(u)$ is *typ* complex valued,

Def **Ampl. Spec** $A(u) = ||F(u)||$

Def **Phase Spec** $\phi(u) = \arg F(u) = \arctan\left(\frac{\Im(F(u))}{\Re(F(u))}\right)$

Def **DFT** used for analyzing discrete data. Applies base change to values via matrix multiplication. More general:

$$F(k) = \sum_{n=0}^{N-1} x_n e^{-i2\pi k n \div N} \text{ for } k \in \{0, 1, \dots, N-1\}$$

For images at location (u, v), or $F = Uf$, with U Fourier Mat.:

$$F(u,v) = \sum_{x=0}^{N-1} \sum_{y=0}^{M-1} I(x,y) \cdot e^{-i2\pi\left(\frac{ux}{N} + \frac{vy}{M}\right)}$$

For continuous time, we get:

$$\mathcal{F}[g(x,y)](u,v) = \int \int_{\mathbb{R}^2} g(x,y) e^{-2i\pi(ux+vy)} \, \mathrm{d}x \, \mathrm{d}y$$

Inverted by negating exp and swapping g and F (or F and I)

Thm **Convolution** $\mathcal{F}[f * g] = F \cdot G$, $\mathcal{F}[f \cdot g] = F * G$, with F, G the fourier transforms

Rem **Filtering** Can be achieved: $h * f = \mathcal{F}^{-1}[H \cdot F]$

Rem **Dirac Delta** $\delta(x) = \begin{cases} \infty & \text{if } x = 0 \\ 0 & \text{otherwise} \end{cases}$, following prop for sampling to alleviate issue of 0 everywhere after fourier:

$$\int_{-\infty}^{\infty} f \cdot \delta(x - a) \, \mathrm{d} = f(a)$$

Rem **Restoration** To restore image, inverse kernel $\widetilde{h}(x) = \mathcal{F}^{-1}\left[\frac{1}{H(x)}\right]$, with $H(x) = \mathcal{F}[h(x)]$ and $h(x)$ degradation filter.

Ex For motion blur: $h(x,y) = \frac{1}{2l}[\theta(x+l) - \theta(x-l)]\delta(y)$. $H(x) = \frac{\sin(2\pi ul)}{2\pi ul}$

1.5 Local Features

1.6 Optical Flow

Def **Brightness** $I(x,y) = 0.299 \cdot R(x,y) + 0.587 \cdot G(x,y) + 0.114 \cdot B(x,y)$, in video $I(x,y,y)$ at given timestamp

Def **Flow Visualization** (Will add colour wheel here in some way, slide 43 of optical flow slide deck)

Rem **Optical Flow** estimates motion of each pixel in consecutive frames. We look for nearby pixels of same colour. For that, assume small motion (only moves a bit) and colour consistency (i.e. brightness stays consistent for same pixel)

Def **Bright. Constancy Eq.** $\frac{\partial I}{\partial x}\frac{\mathrm{d}x}{\mathrm{d}t} + \frac{\partial I}{\partial y}\frac{\mathrm{d}y}{\mathrm{d}t} + \frac{\partial I}{\partial x} = 0$. Derived from multivariate Taylor expansion, cancelling terms, dividing by δt. Shorthand: $\mathbb{I}_x u + I_y v + I_t = 0$, or in vector form: $\nabla I^\top v + I_t = 0$. I_x, I_y are image gradients at point p, u, x are flow velocities.

Comp: I_x, I_y) via Sobel Filter or DoG., I_t via frame diff. u and v need to be solved for (unknowns in optical flow problem). Solution not unique with single pixel, lays on line:

Def **Lucas-Kanade** Flow locally smooth, const. displ. between pixels. Equiv to solving SLE $A^\top A \cdot \widehat{x} = A^\top b$

$$\begin{bmatrix} \sum_{p\in P} I_x I_x & \sum_{p\in P} I_x I_y \\ \sum_{p\in P} I_y I_x & \sum_{p\in P} I_y I_y \end{bmatrix} \cdot \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} \sum_{p\in P} I_x I_t \\ \sum_{p\in P} I_y I_t \end{bmatrix}$$

$\Rightarrow$ Harris Corner Detector (corners: good place to comp flow). If rank of $A < 2$, then aperture problem: (visible part of image too small to tell movement dir).

Def **Horn-Schunck**

$$E[u,v] = \iint_\Omega [(I_x u + I_y v + I_t)^2 + \lambda(||\nabla u||^2 + ||\nabla v||^2)] \, \mathrm{d}x \, \mathrm{d}y$$

w/ $||\nabla u||^2 = \left(\frac{\partial u}{\partial x}\right)^2 + \left(\frac{\partial u}{\partial y}\right)^2$, λ weight. In practice: img grid

$$\min_{u,v} \sum_{i,j} E_s(i,j) + \lambda E_d(i,j)$$

Algorithm 1 Horn-Schunck

1 Precompute image gradients I_x, I_y
2 Precompute temporal gradients I_t
3 Initialize flow-field $(u = 0, v = 0)$
4 **while** not converged **do**
5 Compute flow field updates for each pixel $\widehat{u}_{kl} = \overline{u}_{kl} - Q \cdot I_x$, $\widehat{v}_{kl} = \overline{v}_{kl} - Q \cdot I_y$

with $Q = \frac{I_x \overline{u}_{kl} + I_y \overline{v}_{kl} + I_t}{\lambda^{-1} + I_x^2 + I_y^2}$, with $\overline{u}_{kl}$ prev value, $\widehat{u}_{kl}$ new value.